

Міністерство освіти і науки України
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ім. Богдана Хмельницького

Факультет Обчислювальної техніки, інтелектуальних та
управляючих систем
Кафедра Програмного забезпечення автоматизованих систем

ЛАБОРАТОРНА РОБОТА №5
по дисципліні «Системне програмування»

**Тема: МОДЕЛЮВАННЯ І УСУНЕННЯ СИТУАЦІЙ ВЗАЄМНОГО
БЛОКУВАННЯ І ГОЛОДУВАННЯ**

Виконала: студентка гр. КС-24
Гук М.О.

Перевірив: викладач
кафедри ІТ
Авраменко А.С.

Черкаси, 2026

Мета роботи: вивчити принципи виникнення взаємоблокувань і реалізувати моделювання алгоритму банкіра для перевірки безпечності стану системи.

ЗАВДАННЯ

Розробити програму підвищеної складності, що моделює роботу алгоритму банкіра. Кількість процесів має задаватися в межах від 2 до 4, кількість типів ресурсів - від 1 до 3.

- згенерувати випадкову систему процесів і ресурсів у заданих межах;
- сформувати матриці Allocation, Max і Need;
- обчислити вектор доступних ресурсів Available;
- перевірити стан системи алгоритмом банкіра;
- вивести безпечну послідовність або показати, що стан небезпечний.

ХІД РОБОТИ

Посилання на гіт: https://github.com/HukMariia1124/SystemProgramming_Lab5

Програма реалізована мовою C# з графічним інтерфейсом WPF. Інтерфейс дозволяє генерувати параметри системи, переглядати матриці ресурсів і запускати аналіз алгоритмом банкіра.

У програмі використовуються такі основні позначення:

Таблиця 1 – Поля програми

Позначення	Поле в програмі	Зміст
E	_total	загальна кількість ресурсів у системі
A / Available	_available	кількість ресурсів, які зараз вільні
C / Allocation	_allocation	ресурси, вже виділені процесам
Max	_maximum	максимальні потреби

		процесів
Need	_need	ресурси, які ще можуть знадобитися процесу
Work	work	робоча копія Available для уявного завершення процесів

Матриця потреб обчислюється за формулою:

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j];$$

Вектор доступних ресурсів обчислюється як різниця між загальною кількістю ресурсу і сумою виділених одиниць цього ресурсу:

$$\text{Available}[j] = \text{Total}[j] - \text{sum}(\text{Allocation}[i, j]);$$

Основні можливості інтерфейсу програми:

- випадкова генерація кількості процесів від 2 до 4;
- випадкова генерація кількості типів ресурсів від 1 до 3;
- перегляд і редагування матриць Allocation і Max;
- автоматичний перерахунок Need і Available;
- покроковий лог роботи алгоритму банкіра.

Генерація стану системи виконується в методі GenerateScenario. Спочатку створюється матриця Allocation, після цього для кожного процесу формується Max, а Need рахується як різниця між Max і Allocation.

Скорочений фрагмент генерації стану системи:

```
// Генерація Max і Need після випадкового Allocation.
for (var i = 0; i < processCount; i++)
{
    for (var j = 0; j < resourceCount; j++)
    {
        var extraNeed = _random.Next(0, Math.Max(2, _total[j] / 2) + 1);
        _maximum[i, j] = Math.Min(_total[j], _allocation[i, j] + extraNeed);
        _need[i, j] = _maximum[i, j] - _allocation[i, j];
    }
}
```

```
    }  
}  
  
RecalculateDerivedState();  
BindAllGrids();
```

Перевірка безпечності виконується методом `CheckSafety`. Алгоритм бере `Work = Available` і шукає процес, для якого `Need` не перевищує `Work`. Якщо такий процес існує, він уявно завершується і повертає виділені ресурси в `Work`.

```
var work = (int[])_available.Clone();  
var finish = new bool[processCount];  
var sequence = new List<int>();  
  
while (changed)  
{  
    changed = false;  
    for (var i = 0; i < processCount; i++)  
    {  
        if (finish[i] || !RowLessOrEqual(_need, i, work))  
        {  
            continue;  
        }  
  
        for (var j = 0; j < resourceCount; j++)  
        {  
            work[j] += _allocation[i, j];  
        }  
  
        finish[i] = true;  
        sequence.Add(i);  
        changed = true;  
    }  
}
```

Якщо всі процеси вдалося позначити як завершені, стан вважається безпечним. Якщо на певному кроці немає жодного процесу, для якого виконується $Need \leq Work$, стан є небезпечним. Приклади роботи:

The screenshot shows the application interface with the following data:

- Parameters:** 4 processes, 3 types of resources, total resources [8, 10, 9].
- Allocated (C):**

	R1	R2	R3
P0	1	0	2
P1	4	0	4
P2	1	1	1
P3	2	4	0
- Maximum (Max):**

	R1	R2	R3
P0	5	0	6
P1	4	5	7
P2	3	3	3
P3	5	8	0
- Needs (Need = Max - C):**

	R1	R2	R3
P0	4	0	4
P1	0	5	3
P2	2	2	2
P3	3	4	0
- Log:** The log indicates an unsafe state where processes P0, P1, and P3 cannot finish without resources.

Рис. 1 – Небезпечний стан

The screenshot shows the application interface with the following data:

- Parameters:** 2 processes, 3 types of resources, total resources [9, 8, 9].
- Allocated (C):**

	R1	R2	R3
P0	1	1	0
P1	1	1	4
- Maximum (Max):**

	R1	R2	R3
P0	4	5	3
P1	4	1	4
- Needs (Need = Max - C):**

	R1	R2	R3
P0	3	4	3
P1	3	0	0
- Log:** The log indicates a safe state where the sequence of completion for processes P0 and P1 is possible.

Рис. 2 – Безпечний стан

ВИСНОВОК

У ході виконання лабораторної роботи було розроблено WPF-програму для моделювання алгоритму банкіра. Програма генерує систему з 2-4 процесів і 1-3 типів ресурсів, формує матриці Allocation, Max і Need, обчислює вектор Available та виконує перевірку стану системи на безпечність. У журналі моделювання покроково відображається робота алгоритму: початковий Work, перевірка умови $Need \leq Work$, уявне завершення процесу та повернення ресурсів. Якщо існує послідовність завершення всіх процесів, стан визначається як безпечний; інакше програма повідомляє про небезпечний стан.